07. Plonky3 라이브러리 상세 분석

7.1 Plonky3 개요

7.1.1 Plonky3란?

Plonky3는 모듈화된 STARK 증명 시스템 라이브러리입니다.

핵심 특징:

- **모듈화**: 각 구성 요소를 독립적으로 교체 가능
- **고성능**: Rust로 작성되어 빠른 실행
- **유연성**: 다양한 필드, 해시 함수, 커밋먼트 스킴 지원
- **타입 안전성**: Rust의 타입 시스템 활용

7.1.2 Plonky3 vs Plonky2

| 특징 | Plonky2 | Plonky3 |

|------|---------|---------|

| 증명 시스템 | PLONK | STARK |

| 모듈화 | 제한적 | 높음 |

| 필드 | Goldilocks | 다양한 필드 |

| 사용 사례 | 특정 용도 | 범용 |

7.2 사용된 Plonky3 크레이트

7.2.1 p3-air

역할: AIR (Algebraic Intermediate Representation) 정의

```
use p3_air::{Air, AirBuilder, BaseAir};
```

BaseAir trait

```
pub trait BaseAir<F: Field> {
fn width(&self) -> usize; // 추적 테이블의 컬럼 수
}
```

구현 예시:

```
impl<F: Field> BaseAir<F> for VMAir {
fn width(&self) -> usize {
11 // 스택 4 + push_val 1 + 플래그 5 + extra_data 1
}
}
```

Air trait

```
pub trait Air<AB: AirBuilder>: BaseAir<AB::F> {
fn eval(&self, builder: &mut AB);
}
```

구현 예시:

```rust
impl<AB: AirBuilder> Air<AB> for VMAir {
fn  eval(&self, builder: &mut AB) {
let  main = builder.main();
let  local = main.row_slice(0);
let  next = main.row_slice(1);
// 제약 조건 정의
builder.when_transition().assert_zero(/* ... */);
}
}
```

AirBuilder

주요 메서드:

- `main()` : 메인 추적 테이블 접근
- `when_first_row()` : 첫 번째 행에서만 적용되는 제약
- `when_transition()` : 상태 전이 제약
- `when_last_row()` : 마지막 행 제약
- `assert_zero(expr)` : 표현식이 0이어야 함을 주장
- `assert_eq(a, b)` : 두 값이 같아야 함을 주장

7.2.2 p3-field

역할: 유한체 연산

```rust
use p3_field::{Field, FieldAlgebra, PrimeField32};
```

Field trait

```
pub trait Field: FieldAlgebra {
const ZERO: Self;
const ONE: Self;
const TWO: Self;
const NEG_ONE: Self;
fn from_canonical_u32(n: u32) -> Self;
fn as_canonical_u32(&self) -> u32;
}
```

사용 예시:

```
let zero = Mersenne31::ZERO;
let one = Mersenne31::ONE;
let ten = Mersenne31::from_canonical_u32(10);
let sum = ten + one; // 11
```

FieldAlgebra trait

```
pub trait FieldAlgebra: Add + Sub + Mul + Div + Neg {
fn inverse(&self) -> Self;
fn square(&self) -> Self;
fn exp_u64(&self, power: u64) -> Self;
```

```
}
```

7.2.3 p3-mersenne-31

역할: Mersenne31 필드 구현

```
use p3_mersenne_31::Mersenne31;
```

Mersenne31 구조

```
pub struct Mersenne31 {
value: u32, // 0 <= value < 2^31 - 1
}
```

특징:

- 소수: $p = 2^{31} - 1 = 2{,}147{,}483{,}647$
- 빠른 모듈로 연산: `x mod p = (x & p) + (x >> 31)`
- 32비트 정수로 표현 가능

연산 예시:

```
let a = Mersenne31::from_canonical_u32(100);
let b = Mersenne31::from_canonical_u32(200);
let sum = a + b; // 300
let product = a * b; // 20000
let quotient = b / a; // 2
```

7.2.4 p3-matrix

역할: 행렬 연산 및 표현

```
use p3_matrix::{Matrix, dense::RowMajorMatrix};
```

RowMajorMatrix

```
pub struct RowMajorMatrix<T> {
values: Vec<T>,
width: usize,
}
```

생성:

```
let values = vec![1, 2, 3, 4, 5, 6];
let matrix = RowMajorMatrix::new(values, 3);
// 행렬:
// [1, 2, 3]
// [4, 5, 6]
```

접근:

```
let row = matrix.row(0); // [1, 2, 3]
let element = matrix.get(1, 2); // 6
```

7.2.5 p3-uni-stark

역할: Univariate STARK 증명 시스템

```
use p3_uni_stark::{prove, verify, StarkConfig};
```

prove 함수

```
pub fn prove<SC, A>(
config: &SC,
air: &A,
challenger: &mut SC::Challenger,
trace: RowMajorMatrix<SC::Val>,
public_values: &[SC::Val],
) -> Proof<SC>
where
SC: StarkConfig,
A: Air<ConstraintBuilder<SC>>,
```

파라미터:

- `config` : STARK 설정
- `air` : AIR 정의
- `challenger` : 도전자 (Fiat-Shamir)
- `trace` : 실행 추적
- `public_values` : 공개 입력/출력

반환: STARK 증명

verify 함수

```
pub fn verify<SC, A>(
config: &SC,
air: &A,
challenger: &mut SC::Challenger,
proof: &Proof<SC>,
public_values: &[SC::Val],
) -> Result<(), VerificationError>
```

반환: 검증 성공 또는 에러

StarkConfig

```
pub struct StarkConfig<Pcs, Challenge, Challenger> {
pcs: Pcs,
_phantom: PhantomData<(Challenge, Challenger)>,
}
```

구성 요소:

- `Pcs` : Polynomial Commitment Scheme
- `Challenge` : 도전 값 타입
- `Challenger` : 도전자 타입

7.2.6 p3-fri

역할: FRI (Fast Reed-Solomon IOP) 프로토콜

```
use p3_fri::FriConfig;
```

FriConfig

```
pub struct FriConfig<M> {
pub log_blowup: usize,
pub num_queries: usize,
pub proof_of_work_bits: usize,
pub mmcs: M,
}
```

파라미터 설명:

log_blowup

```
log_blowup: 1  // blowup = 2^1 = 2
```

- 다항식 평가 도메인 확장 비율
- 낮은 차수 테스트의 정확성 향상

num_queries

```
num_queries: 40
```

- FRI 프로토콜의 쿼리 수
- 보안 수준 결정: ~2^(-num_queries)

proof_of_work_bits

```
proof_of_work_bits: 8
```

- 작업 증명 난이도
- 무차별 대입 공격 방지

7.2.7 p3-circle

역할: Circle STARK PCS

```
use p3_circle::CirclePcs;
```

CirclePcs

```
pub struct CirclePcs<Val, ValMmcs, ChallengeMmcs> {
pub mmcs: ValMmcs,
pub fri_config: FriConfig<ChallengeMmcs>,
pub _phantom: PhantomData<Val>,
}
```

Circle STARK:

- Circle curve 기반 STARK
- 효율적인 다항식 커밋먼트
- FRI와 통합

7.2.8 p3-merkle-tree

역할: 머클 트리 기반 커밋먼트

```
use p3_merkle_tree::MerkleTreeMmcs;
```

MerkleTreeMmcs

```
pub struct MerkleTreeMmcs<T, U, H, C, const DIGEST_ELEMS: usize> {
// T: 데이터 타입
// U: 해시 출력 타입
// H: 해시 함수
// C: 압축 함수
// DIGEST_ELEMS: 다이제스트 크기
}
```

사용 예시:

```
type ValMmcs = MerkleTreeMmcs<
Mersenne31, // 데이터 타입
u8, // 해시 출력 타입
FieldHash, // 해시 함수
MyCompress, // 압축 함수
32 // 다이제스트 크기 (바이트)
>;
```

동작:

```
데이터: [d1, d2, d3, d4]

↓ 해싱

리프: [H(d1), H(d2), H(d3), H(d4)]

↓ 압축

중간: [C(H(d1),H(d2)), C(H(d3),H(d4))]

↓ 압축

루트: C(C(H(d1),H(d2)), C(H(d3),H(d4)))
```

7.2.9 p3-commit

역할: 커밋먼트 스킴 추상화

```rust
use p3_commit::ExtensionMmcs;
```

ExtensionMmcs

```rust
pub  struct ExtensionMmcs<Val, Challenge, InnerMmcs> {
inner: InnerMmcs,
_phantom: PhantomData<(Val, Challenge)>,
}
```

역할:

- 기본 필드 MMCS를 확장 필드로 확장
- FRI에서 확장 필드 사용 시 필요

7.2.10 p3-keccak

역할: Keccak 해시 함수

```
use p3_keccak::Keccak256Hash;
```

Keccak256Hash

```
pub struct Keccak256Hash;

impl Hasher<u8> for Keccak256Hash {
fn hash(&self, input: &[u8]) -> [u8; 32] {
// Keccak-256 해싱
}
}
```

특징:

- SHA-3 표준
- 256비트 출력
- 암호학적으로 안전

7.2.11 p3-symmetric

역할: 대칭 암호 프리미티브

```
use p3_symmetric::{
CompressionFunctionFromHasher,
```

```
SerializingHasher32,
};
```

SerializingHasher32

```
pub struct SerializingHasher32<H> {
hasher: H,
}
```

역할:

- 32비트 필드 원소를 바이트로 직렬화
- 해시 함수로 해싱

사용:

```
let field_hash = SerializingHasher32::new(Keccak256Hash {});
let hash = field_hash.hash(&[
Mersenne31::from_canonical_u32(10),
Mersenne31::from_canonical_u32(20),
]);
```

CompressionFunctionFromHasher

```
pub struct CompressionFunctionFromHasher<H, const N: usize, const DIGEST_ELEMS:
hasher: H,
```

```
}
```

역할:

- N개의 해시 값을 하나로 압축
- 머클 트리 구성에 사용

사용:

```
type MyCompress = CompressionFunctionFromHasher<Keccak256Hash, 2, 32>;
let compress = MyCompress::new(Keccak256Hash {});
let compressed = compress.compress([hash1, hash2]);
```

7.2.12 p3-challenger

역할: Fiat-Shamir 변환

```
use p3_challenger::{HashChallenger, SerializingChallenger32};
```

HashChallenger

```
pub struct HashChallenger<T, H, const N: usize> {
hasher: H,
state: Vec<T>,
}
```

메서드:

- `observe(value)` : 값 관찰 (상태에 추가)

- sample() : 도전 값 샘플링

동작:

```
let mut challenger = HashChallenger::new(Keccak256Hash {});
challenger.observe(commitment);
let challenge = challenger.sample(); // Hash(commitment)
```

SerializingChallenger32

```
pub struct SerializingChallenger32<Val, Inner> {
inner: Inner,
_phantom: PhantomData<Val>,
}
```

역할:

- 32비트 필드 원소를 직렬화하여 도전자에 전달

7.3 타입 시스템 통합

7.3.1 타입 별칭 체인

```
// 기본 필드
type Val = Mersenne31;

// 확장 필드
```

```
type Challenge = BinomialExtensionField<Val, 3>;


// 해시 함수
type ByteHash = Keccak256Hash;
type FieldHash = SerializingHasher32<ByteHash>;


// 압축 함수
type MyCompress = CompressionFunctionFromHasher<ByteHash, 2, 32>;


// MMCS
type ValMmcs = MerkleTreeMmcs<Val, u8, FieldHash, MyCompress, 32>;
type ChallengeMmcs = ExtensionMmcs<Val, Challenge, ValMmcs>;


// PCS
type Pcs = CirclePcs<Val, ValMmcs, ChallengeMmcs>;


// Challenger
type Challenger = SerializingChallenger32<Val, HashChallenger<u8, ByteHash, 32>>


// STARK Config
type MyConfig = StarkConfig<Pcs, Challenge, Challenger>;
```

7.3.2 타입 의존성 그래프

```
Mersenne31 (Val)
↓
BinomialExtensionField<Val, 3> (Challenge)
↓
Keccak256Hash (ByteHash)
↓
SerializingHasher32<ByteHash> (FieldHash)
↓
CompressionFunctionFromHasher<ByteHash, 2, 32> (MyCompress)
↓
MerkleTreeMmcs<Val, u8, FieldHash, MyCompress, 32> (ValMmcs)
↓
ExtensionMmcs<Val, Challenge, ValMmcs> (ChallengeMmcs)
↓
CirclePcs<Val, ValMmcs, ChallengeMmcs> (Pcs)
↓
SerializingChallenger32<Val, HashChallenger<...>> (Challenger)
↓
StarkConfig<Pcs, Challenge, Challenger> (MyConfig)
```

7.4 Plonky3의 모듈화 설계

7.4.1 교체 가능한 구성 요소

필드 교체

```
// Mersenne31 대신 Goldilocks 사용
type Val = Goldilocks;
// 나머지 코드는 동일하게 동작
```

해시 함수 교체

```
// Keccak 대신 Blake3 사용
type ByteHash = Blake3Hash;
// 나머지 코드는 동일하게 동작
```

PCS 교체

```
// CirclePcs 대신 다른 PCS 사용
type Pcs = AnotherPcs<Val, ValMmcs, ChallengeMmcs>;
```

7.4.2 모듈화의 장점

1. **유연성**: 요구사항에 맞게 구성 요소 선택
2. **실험**: 다양한 조합 테스트
3. **최적화**: 특정 환경에 최적화된 구현 선택
4. **확장성**: 새로운 구성 요소 쉽게 추가

7.5 성능 최적화

7.5.1 Mersenne31의 효율성

```
// 빠른 모듈로 연산
impl Mersenne31 {
fn  reduce(x: u64) -> u32 {
let (lo, hi) = (x  as u32, (x >> 31) as u32);
let  sum = lo + hi;
if  sum >= Self::ORDER {
sum - Self::ORDER
} else {
sum
}
}
}
```

비교:

- 일반 모듈로: `x % p` (느림)
- Mersenne31: `(x & p) + (x >> 31)` (빠름)

7.5.2 병렬 처리

Plonky3는 다음을 병렬화합니다:

- 다항식 평가
- 머클 트리 구성
- FRI 폴딩

```
// Rayon을 사용한 병렬 처리 (내부 구현)
values.par_iter().map(|v| hash(v)).collect()
```

7.5.3 메모리 최적화

- 스트리밍 방식의 데이터 처리
- 불필요한 복사 최소화
- 효율적인 메모리 레이아웃

7.6 Plonky3 사용 패턴

7.6.1 기본 패턴

```
// 1. 타입 정의
type Val = Mersenne31;
type Challenge = BinomialExtensionField<Val, 3>;

// 2. 구성 요소 설정
let  hash = Keccak256Hash {};
let  mmcs = MerkleTreeMmcs::new(/* ... */);
let  fri_config = FriConfig { /* ... */ };
let  pcs = CirclePcs { /* ... */ };

// 3. STARK 설정
```

```rust
let  config = StarkConfig::new(pcs);


// 4. 증명 생성
let  proof = prove(&config, &air, &mut  challenger, trace, &[]);


// 5. 증명 검증
verify(&config, &air, &mut  challenger, &proof, &[]);
```

7.6.2 커스터마이징 패턴

```rust
// 커스텀 필드
struct MyField { /* ... */ }
impl Field for MyField { /* ... */ }


// 커스텀 AIR
struct MyAir { /* ... */ }
impl<AB: AirBuilder> Air<AB> for MyAir { /* ... */ }


// 커스텀 설정으로 증명 생성
let  proof = prove(&my_config, &my_air, &mut  challenger, trace, &[]);
```

7.7 요약

Plonky3는 강력하고 유연한 STARK 라이브러리입니다:

1. **모듈화**: 각 구성 요소를 독립적으로 교체 가능
2. **타입 안전성**: Rust의 타입 시스템 활용
3. **고성능**: 최적화된 구현
4. **확장성**: 새로운 기능 쉽게 추가

이 라이브러리를 사용하면 복잡한 STARK 증명 시스템을 비교적 쉽게 구축할 수 있습니다.